

Relazione per
“Programmazione ad Oggetti”

CardHub

Flavio Ciliberti
flavio.ciliberti@studio.unibo.it

Kaan Portolani
kaan.portolani@studio.unibo.it

Tobias Basilisco
tobias.basilisco@studio.unibo.it

13 settembre 2026

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.2	Modello del Dominio	4
2	Design	8
2.1	Architettura	8
2.2	Design dettagliato	9
2.2.1	Disaccoppiamento Controller-View (Basilisco)	9
2.2.2	Variabilità delle modalità di partita	11
2.2.3	Interfacce generiche per le schermate CRUD	13
2.2.4	Riuso e corenze visiva nei componenti Swing	16
2.2.5	Caricamento lazy e cache delle immagini delle carte (ImageResolver)	18
2.2.6	Creazione e caricamento dei mazzi (Portolani)	20
2.2.7	Costruzione flessibile delle carte	20
2.2.8	Gestione della logica di gioco (Ciliberti)	22
2.2.9	Incapsulamento del Player	23
2.2.10	Inizializzazione della partita	24
3	Sviluppo	25
3.1	Testing automatizzato	25
3.2	Note di sviluppo	26
3.2.1	Basilisco	26
3.2.2	Portolani	27
3.2.3	Ciliberti	28
4	Commenti finali	29
4.1	Autovalutazione e lavori futuri	29
4.1.1	Basilisco	29
4.1.2	Portolani	30
4.1.3	Ciliberti	31

A Guida utente	32
B Esercitazioni di laboratorio	33
B.1 tobias.basilisco@studio.unibo.it	33

Capitolo 1

Analisi

1.1 Descrizione e requisiti

L'applicazione consiste in un semplice motore per giochi di carte a due giocatori. L'utente dispone di due opzioni principali di gioco: la prima modalità consiste in un simulatore libero di giochi di carte in cui l'utente (dopo aver scelto un mazzo con cui giocare) è libero di fare ciò che vuole con le sue carte, e la partita si conclude quando uno dei due giocatori decide di arrendersi. L'utente può scegliere se giocare con parametri già stabiliti o se modificarne alcuni, come la dimensione massima della mano e del terreno di gioco.

La seconda modalità di gioco consiste in una versione modificata del gioco Carta-E apparso nella famosa serie di animazione giapponese *Gyakkyou Burai Kaiji*, una sorta di versione modificata di sasso-carta-forbici, dove una volta esaurite le carte il vincitore viene determinato dal sistema in base a chi dei due giocatori ha ottenuto più punti.

Requisiti funzionali

- L'utente dovrà essere in grado di creare una partita scegliendo i mazzi, la modalità gioco, e quando previsto, i parametri della partita.
- I giocatori dovranno essere in grado di compiere le azioni più importanti nei giochi di carte: pescare dal deck, giocare carte dalla mano, dichiarare un attacco ad una carta dell'avversario e spostare le carte nella pila degli scarti.
- Il giocatore deve essere in grado di visualizzare le informazioni delle carte che ha in mano prima di decidere se giocarle o meno.

- L'applicazione dovrà implementare un modo per permettere ad entrambi i giocatori di giocare a turno sullo stesso schermo senza particolari problemi (ad esempio impedire che il giocatore di turno possa vedere la mano dell'avversario).

Requisiti non funzionali

- L'interfaccia dell'applicazione dovrà essere *user-friendly*: la navigazione dei menu dovrà essere intuitiva, e l'interfaccia dovrà essere in grado di adattarsi ad eventuali ridimensionamenti della finestra da parte dell'utente.
- L'interfaccia dovrà consentire agli utenti di comprendere facilmente lo stato della partita e le azioni disponibili, e dovrà notificare opportunamente il giocatore in caso di mosse non consentite durante la partita, o in caso di errori durante la creazione della partita.
- Lo stato mostrato ai giocatori deve essere coerente con l'effettivo svolgimento della partita.
- Il sistema deve poter supportare l'introduzione futura di nuove tipologie di carte, mazzi o modalità di gioco.

1.2 Modello del Dominio

Il modello del dominio di CardHub è costituito da una partita tra due giocatori che utilizzano delle carte appartenenti a differenti mazzi. Durante la partita, le carte vengono spostate tra diverse aree di gioco e possono essere confrontate secondo le regole previste dalla modalità selezionata.

L'entità centrale del dominio è la **Partita**, che possiede uno stato che comprende i due **Giocatori** partecipanti, il **Campo di gioco** e l'eventuale risultato finale.

Il campo di gioco rappresenta l'area nella quale i giocatori collocano le carte selezionate per compiere **Azioni** previste durante il turno.

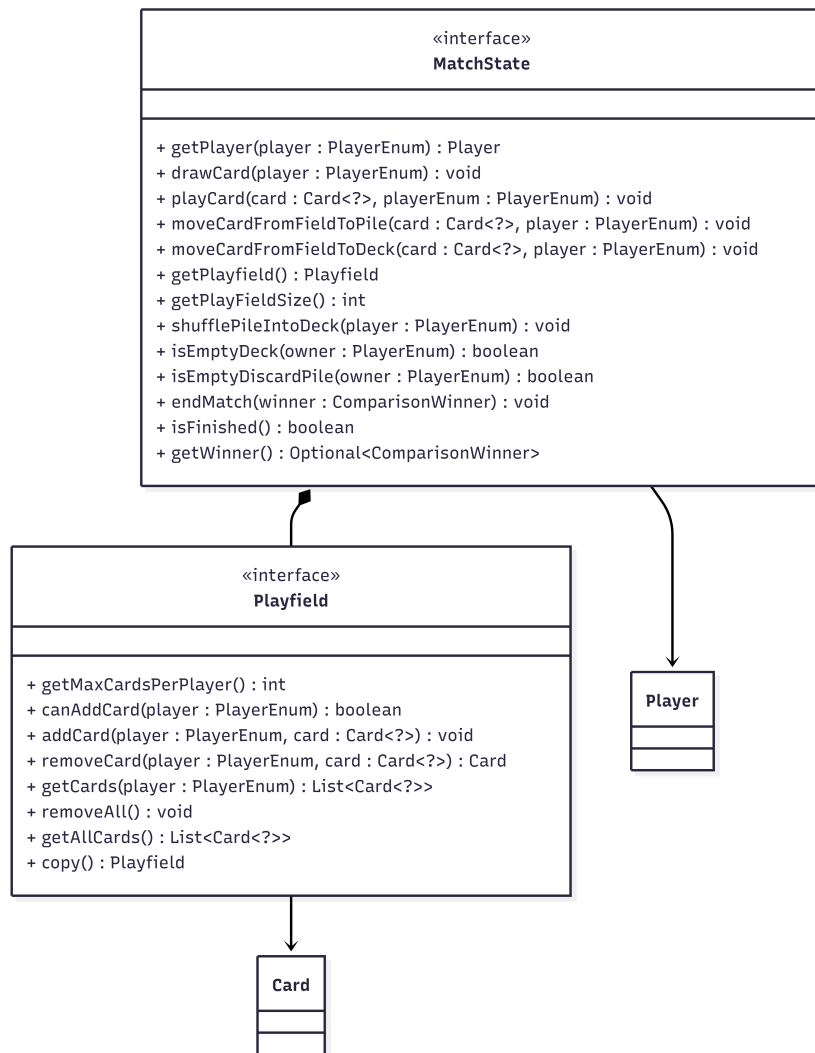


Figura 1.1: Schema UML di MatchState e del Playfield

Ogni giocatore possiede un **Mazzo**, dal quale può ottenere nuove carte durante la partita, una **Mano**, contenente le carte attualmente disponibili per essere giocate, e una **Pila degli scarti**, contenente le carte che sono state scartate o rimosse dal campo di gioco secondo le regole della partita. Il mazzo, la mano e la pila degli scarti sono quindi tutte **Collezioni di carte**, logicamente distinte nel contesto della partita.

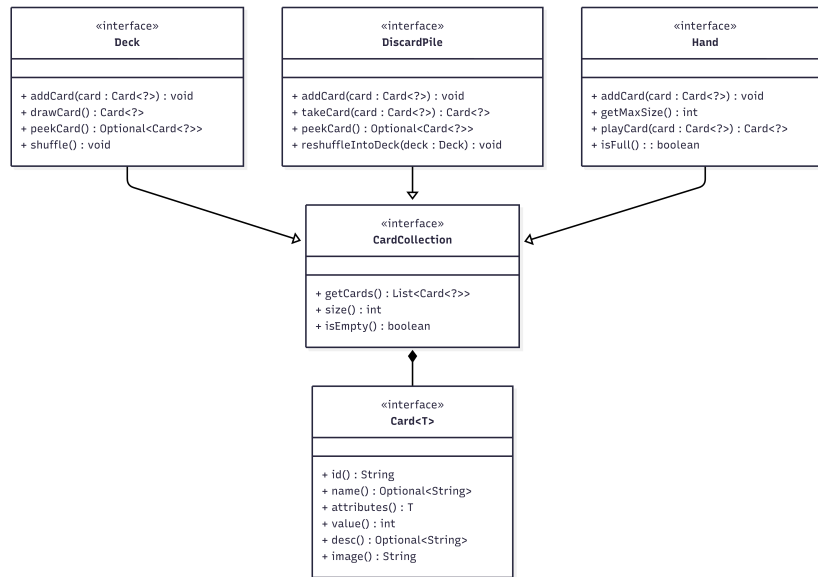


Figura 1.2: Schema UML della struttura di CardCollection

Le **Carte** rappresentano gli elementi fondamentali utilizzati dai giocatori durante una partita. Ogni carta possiede caratteristiche che dipendono dalla tipologia di mazzo a cui appartiene. Tali caratteristiche possono essere utilizzate durante il Confronto con altre carte, il quale produce un risultato che può identificare una carta come vincitrice del confronto oppure determinare un pareggio.

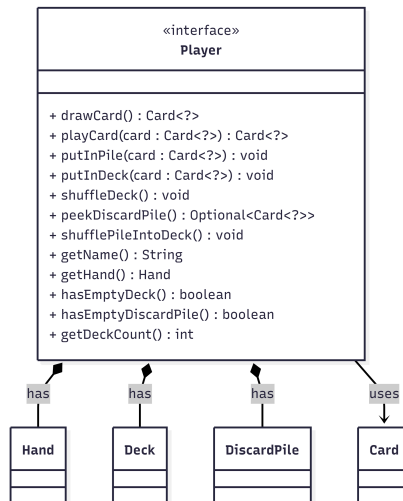


Figura 1.3: Schema UML del Player

L'aspetto più rilevante del dominio consiste quindi nella presenza di differenti tipologie di carte e di differenti regole di gioco. Ciò significa che il dominio deve rappresentare sia gli elementi comuni a tutte le partite sia le caratteristiche che distinguono le diverse modalità di gioco.

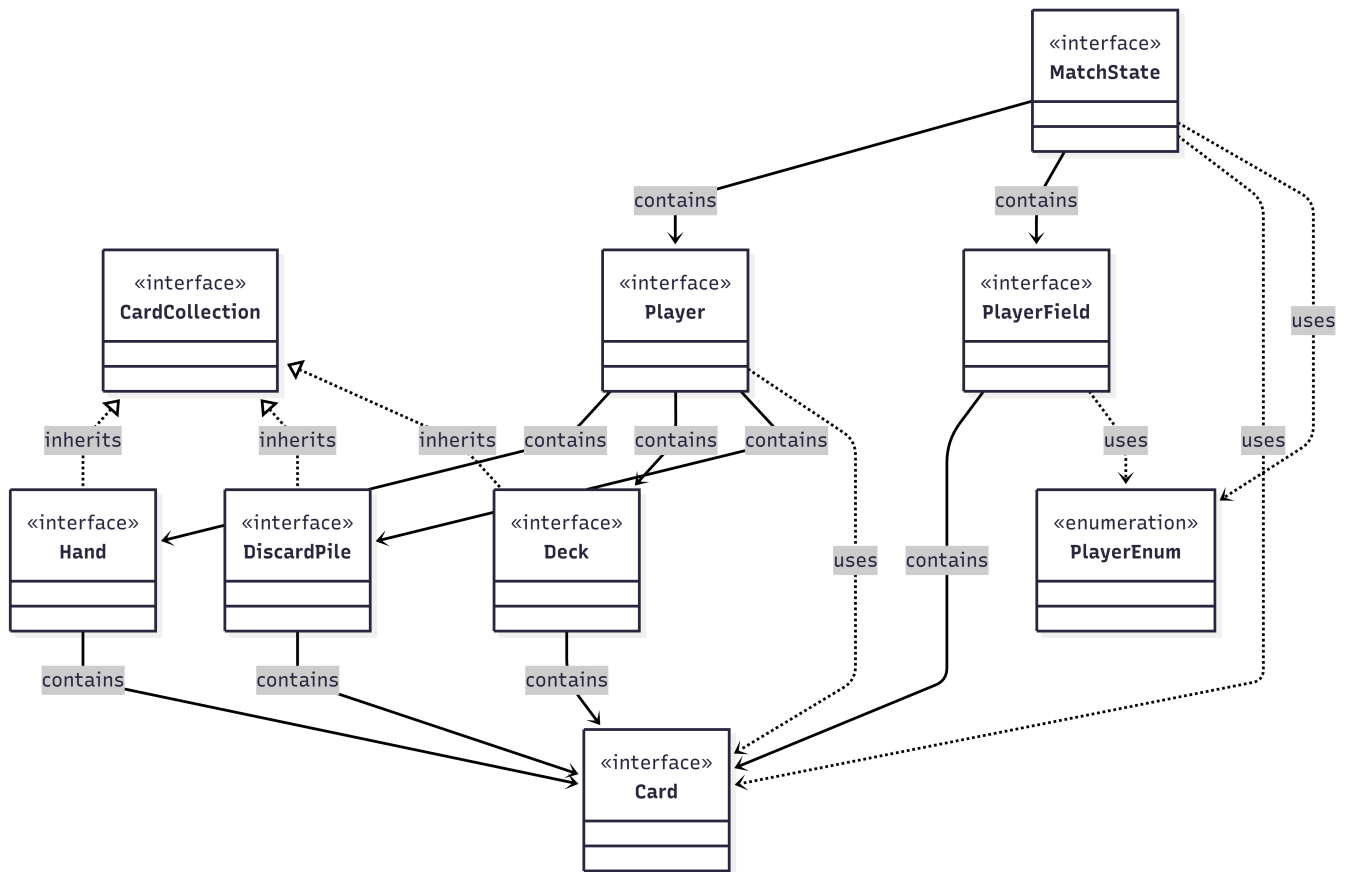


Figura 1.4: Schema UML completo del dominio

Capitolo 2

Design

2.1 Architettura

CardHub adotta il pattern architetturale Model-View-Controller. Ogni schermata dell'applicazione è gestita da una tripletta Controller-View-Model i cui punti di ingresso sono interfacce.

Il contratto comune a tutti i controller è **ScreenController** (*showScreen()*); le schermate che prevedono un ritorno a quella precedente estendono **BackNavigableScreen**, che aggiunge *goBack()*. La navigazione tra le view è centralizzata nell'interfaccia **Navigator** (*show(ScreenId, ScreenView)*, *goHome()*), implementata da **CardLayoutNavigator** sopra uno Swing *CardLayout*; ogni controller la referencia come servizio condiviso (aggregazione), senza possederla in via esclusiva.

Ogni controller crea e possiede (composizione) la propria view attraverso un'interfaccia dedicata, e referencia (aggregazione) il proprio model, quando presente:

- **HomeController - HomeView**: non ha nessun model proprio, coordina solo la navigazione iniziale e l'uscita dall'applicazione;
- **CreateMatchController - CreateMatchView - CreateMatchModel**: quest'ultimo espone i vincoli di dominio (dimensioni minime/massime di mano e campo, mazzi disponibili) per la creazione di una partita;
- **MatchController - MatchView - MatchState/MatchLogic**: queste ultime due interfacce compongono il modello della schermata della partita e sono state trattate in 1.2 e 2.2.8, rispettivamente.
- **ManageDecksController, DeckManagerController, CardManagerController**: seguono la stessa convenzione (un model dedicato per

schermata), ma view e model non sono ancora implementati al momento della stesura di questa relazione; sono comunque riportati nello schema architetturale, tratteggiati, in quanto previsti dalla convenzione adottata.

Grazie a questa struttura, controller e model non dipendono mai dall'implementazione concreta della view, ma solo dalla sua interfaccia: sostituire il toolkit grafico (ad esempio passando da Swing a JavaFX) richiederebbe di riscrivere solo le implementazioni delle view e del **Navigator**, senza alcun impatto su controller e model (si veda il dettaglio in 2.2.1).

Lo schema UML in Figura 2.1 riporta solo le interfacce sopra descritte e le relazioni fra loro.

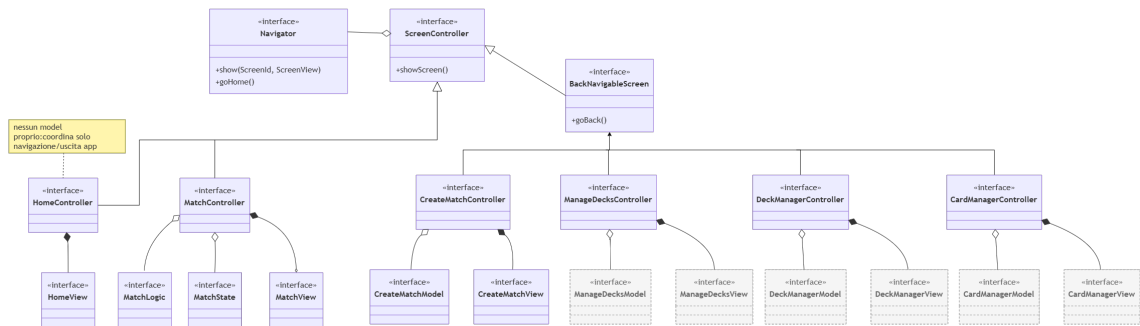


Figura 2.1

2.2 Design dettagliato

2.2.1 Disaccoppiamento Controller-View (Basilisco)

Problema

Il controller deve poter notificare la view dei cambiamenti di stato senza dipendere dalla sua implementazione concreta, in modo che sostituire il toolkit grafico (o la sola resa grafica di una schermata) non richieda modifiche al controller né al model.

Soluzione

Ogni view è definita da un'interfaccia composta dei soli metodi **update*/show*** (es. **MatchView.updateShowingHand(...)**, **updatePlayfield(...)**, **showCurrentPlayer(...)**). Questi metodi vengono invocati dal controller dopo ogni modifica al model per allineare la view allo stato corrente. Il controller crea la propria view concreta (es. **AbstractMatchController.createView()** istanzia **MatchViewImpl**) e la possiede per l'intero ciclo di vita. Nel verso

opposto, l'inoltro delle azioni dell'utente avviene tramite un riferimento diretto e fisso che la view concreta tiene verso il controller che l'ha creata (es. **HomeViewImpl** richiama **HomeController.newMatch()** al click del bottone). La relazione controller-view è quindi bidirezionale ma sempre 1:1: nessuna delle due parti conosce l'altra se non attraverso la rispettiva interfaccia.

Questa soluzione non corrisponde al pattern Observer del GoF, che prevede un soggetto osservato con una lista dinamica di osservatori che si iscrivono/disiscrivono: qui il legame è singolo e fisso alla creazione dell'interfaccia della view che espone solo i metodi di aggiornamento. Come alternativa è stato valutato un vero Observer con lista di listener sulla view, scartato perché ogni schermata ha esattamente una view attiva alla volta: la complessità di gestire iscrizione/rimozione di osservatori non avrebbe portato benefici concreti.

Il model, quando presente, è iniettato dall'esterno nel costruttore del controller (es. **MatchState/MatchLogic** sono create da **MatchControllerFactory**, trattato in 2.1.1, e passate ad **AbstractMatchController**): il controller lo referencia (aggregazione) ma non lo crea; a differenza della view, non esiste alcuna dipendenza di ritorno dal model verso il controller.

Lo schema generico in Figura 2.2 riassume la relazione fra **XController/XView/XModel** di una schermata "X" e le rispettive implementazioni.

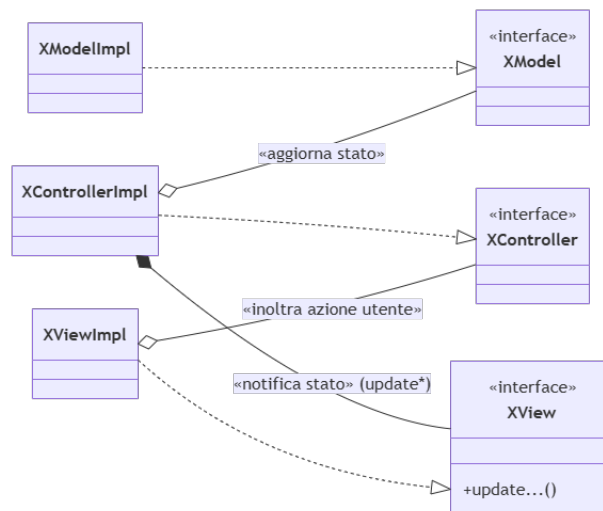


Figura 2.2

2.2.2 Variabilità delle modalità di partita

Problema

CardHub supporta più modalità di partita: libera/personalizzata ed "e-card" che condividono la stessa meccanica di turno (spostamento carte, avanzamento al giocatore successivo). Queste però, differiscono nel comportamento di fine turno e nei dati che la view deve mostrare: la modalità e-card confronta a fine turno le carte giocate e mostra un punteggio, cosa che la modalità standard non fa. Occorre riusare la meccanica comune senza duplicarla per ogni modalità e, allo stesso tempo, permettere a ciascuna modalità di accedere senza downcast alla view e alla logica arricchite delle proprie funzionalità (es. *updateScore()*, *getPoints()*), mantenendo il motore comune disaccoppiato dalle modalità specifiche.

Soluzione

AbstractMatchController<**V extends MatchView**, **L extends MatchLogic**> implementa **MatchController** e realizza *endTurn()* come metodo template: dopo aver nascosto la mano del giocatore di turno, invoca il metodo astratto *onEndTurn()* e infine notifica alla view il cambio di giocatore. **MatchControllerImpl** (**V=MatchView**, **L=MatchLogic**) implementa *onEndTurn()* con la sola logica standard (*logic.changeTurn()*); **ECardMatchController** (**V=ECardMatchView**, **L=ScoredMatchLogic**) invece la implementa con la logica e-card (nasconde il playfield a metà scambio, confronta le carte tramite *compareCard()* e per ultimo aggiorna il punteggio con *updateScore()*).

Ogni sottoclasse crea la propria view concreta tramite il metodo astratto *createView()* (Factory Method), invocato una sola volta nel costruttore di **AbstractMatchController** (annotato *@SuppressWarnings*, poiché la chiamata a un metodo overrideabile nel costruttore è qui intenzionale e sicura). L'implementazione di *createView()* da parte della sottoclasse **MatchControllerImpl** istanzia **MatchViewImpl**, mentre **ECardMatchController**.*createView()* istanzia **ECardMatchViewImpl**. I due parametri generici (**V extends MatchView**, **L extends MatchLogic**) permettono ai metodi protetti *getMatchView()/getLogic()* di restituire il tipo specifico della sottoclasse anziché il tipo base: **ECardMatchController** può quindi chiamare *getMatchView().updateScore(...)* e *getLogic().getPoints(...)*, metodi che esistono solo su **ECardMatchView/ScoredMatchLogic**, senza alcun cast esplicito.

Per quanto riguarda le view, la gerarchia di interfacce

MatchView→**ScoredMatchView**→**ECardMatchView** rispecchia esat-

tamente questa progressione di funzionalità (punteggio, poi anche il pop-up di confronto).

Come alternativa è stata valutata un'unica implementazione di `MatchController`, parametrizzata con un flag o un *enum* di modalità e un blocco condizionale in `onEndTurn()`. Questa opzione è stata scartata perché concentrerebbe in un'unica classe logiche eterogenee (destinate a crescere con l'aggiunta di nuove modalità), e soprattutto perché costringerebbe ad utilizzare view e logica solo di tipo base **MatchView/MatchLogic**, obbligando a downcast espliciti (o a *instanceof*) ogni volta che una modalità necessita di funzionalità proprie come `updateScore()`. La soluzione a sottoclassi con generics evita entrambi i problemi, al costo di una classe in più per ogni nuova modalità: costo ritenuto accettabile, perché ogni modalità corrisponde comunque a un insieme di responsabilità (view, logica di confronto) già naturalmente distinto.

Lo schema UML in Figura 2.3 mostra **AbstractMatchController<V,L>**, le due sottoclassi con il relativo binding dei generics, e le gerarchie di **MatchView/MatchLogic** coinvolte nel binding.

Pattern

Template Method: il metodo template è `endTurn()` (concreto, in **AbstractMatchController**), l'hook è `onEndTurn()` (astratto, ridefinito da **MatchControllerImpl** ed **ECardMatchController**).

Factory Method: l'hook è `createView()` (astratto, invocato nel costruttore), le due sottoclassi fungono da ConcreteCreator e restituiscono i rispettivi ConcreteProduct (**MatchViewImpl**, **ECardMatchViewImpl**).

I generics V e L non sono di per sé un design pattern, ma sono il meccanismo linguistico che rende possibile evitare downcast nell'accesso a view e logica dal Factory Method: per questo vengono presentati in accompagnamento ai due pattern, non come un terzo pattern a sé stante.

nato di elementi, crearne uno nuovo, modificarne o eliminarne uno esistente. Occorre esprimere questo contratto una sola volta, così che le schermate che lo condividono possano essere trattate in modo uniforme. Ciò evita di duplicare le stesse firme di metodo su più interfacce e non costringe a schermate con esigenze diverse ad aderirvi.

Soluzione

L'interfaccia **ItemPageNavigator** isola la sola paginazione (*changePage*, *previousPage*, *nextPage*). **EntityManager<T>** la estende aggiungendo il contratto CRUD (*createNew*, *edit*, *delete*), parametrico rispetto al tipo dell'elemento su cui si basa. **DeckManagerController** estende **EntityManager<Card>**, mentre **ManageDecksController** estende **EntityManager<Deck>**.

La suddivisione in due interfacce (**ItemPageNavigator** e **EntityManager<T>**), invece di un'unica interfaccia con tutti i metodi, segue il principio di interface segregation: una schermata che si limitasse a sfogliare un elenco senza consentirne la modifica potrebbe dipendere dalla sola paginazione, senza ereditare metodi di scrittura mai usati.

Come alternativa è stata valutata la duplicazione delle firme CRUD direttamente su **DeckManagerController** e **ManageDecksController**, senza un'interfaccia comune. È stata scartata perché impedirebbe di trattare le due schermate in modo polimorfo e perché, con future schermate di gestione, il contratto rischierebbe di divergere tra le interfacce invece di restare garantito da un'unica dichiarazione.

Lo schema UML in Figura 2.4 mostra la generalizzazione

EntityManager<T> → **ItemPageNavigator** e le due realizzazioni **DeckManagerController** (T = Card) / **ManageDecksController** (T = Deck).

Gli wireframe in Figura 2.5 e Figura 2.6 mostrano lo schema progettuale della GUI delle schermate **DeckManager** e **ManageDecks** rispettivamente.

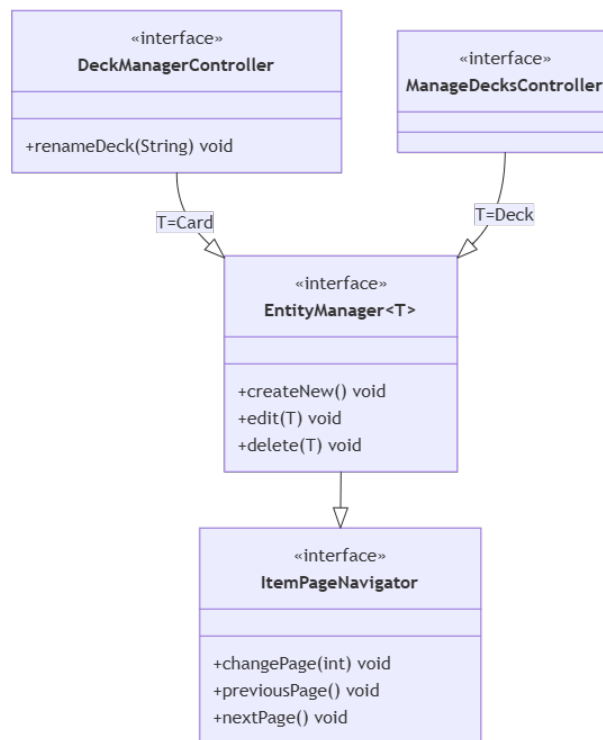


Figura 2.4

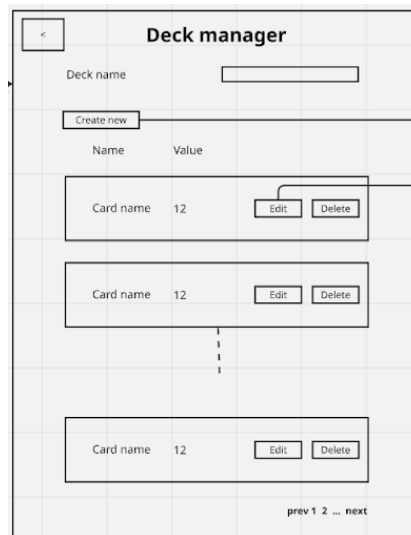


Figura 2.5

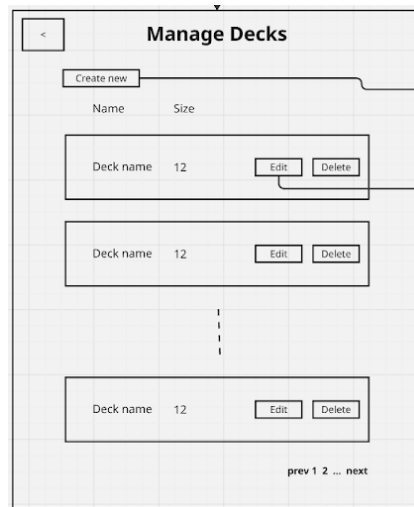


Figura 2.6

2.2.4 Riuso e corenze visiva nei componenti Swing

Problema

CardHub richiede uno stile visivo coerente (colori, font, padding, bordi) su tutte le schermate dell'applicazione. Senza un punto di riferimento centralizzato, ogni view concreta finirebbe per duplicare lo stesso codice di styling e per disseminare nel codice valori "magici" (codici colore esadecimali, dimensioni di padding) sparsi e difficili da mantenere allineati.

Soluzione

Il package `view.components` raccoglie sottoclassi dedicate per ciascun componente grafico di base: **CHButton** extends **JButton**, **CHLabel** extends **JLabel**, **CHTitle** extends **JLabel**, **CHPanel** extends **JPanel**, **ScreenView** extends **JPanel**, **CHTextField** extends **JTextField**, **CHTextArea** extends **JTextArea**, **CHFrame** extends `JFrame`; ciascuna delle quali applica lo stile di default del proprio componente Swing di base.

In **CHButton**, **CHTextArea**, **CHTextField**, **CHTitle** e **ScreenView** la personalizzazione è applicata da un metodo privato `customize()`, invocato dal costruttore; **CHLabel** e **CHPanel** applicano invece lo stile direttamente nei rispettivi costruttori (es. `CHLabel(String, Color)` chiama `setForeground`, `CHPanel(Color)` chiama `setBackground`), senza un proprio metodo `customize()`, ma accettando dei parametri che consentono la creazione di componenti con delle caratteristiche specifiche (le classi clienti utilizzano comunque **CH-Styles** per fare ciò).

Palette e altre costanti di stile sono centralizzate in due classi: **CHColor** è un'enumerazione che associa a ciascun colore dell'applicazione (*PRIMARY*, *SECONDARY*, *TERTIARY*) il proprio codice esadecimale RGB, restituito da *getCode()*; **CHStyles** è una utility class (costruttore privato, che espone i metodi factory *primaryColor*, *secondaryColor*, *tertiaryColor* e le costanti di padding *PADDING_LARGE*, *PADDING_STANDARD*, *PADDING_SMALL*, *PADDING_NONE*). Cambiare la palette dell'applicazione richiede così di modificare un solo punto (**CHColor**), non ogni singola view.

La scelta è di ereditarietà diretta dal componente Swing, non di composizione/wrapping: queste classi sono un dettaglio implementativo interno alle view concrete (le **Impl*), non parte del contratto View esposto al controller (trattato in 2.2.1).

Come alternativa è stato valutato il pattern Decorator, utile a comporre stili dinamicamente a runtime; è stata scartata perché lo stile di ciascun componente è fisso e noto già a compile-time (un **CHButton** è sempre lo stesso tipo di bottone stilizzato), senza necessità di comporre decorazioni diverse sulla stessa istanza.

Nota: Il ricorrere del metodo privato *customize()* in più classi non costituisce un Template Method GoF né un altro pattern noto: sono metodi privati, ripetuti indipendentemente, in classi scorrelate che estendono ciascuna una diversa classe Swing.

Lo schema UML in Figura 2.7 mostra la relazione tra le classi Swing di libreria (**JButton**, **JLabel**, **JPanel**, **JTextField**, **JTextArea**, **JFrame**) e le sottoclassi **CH*** (più **ScreenView**), e le loro dipendenze.

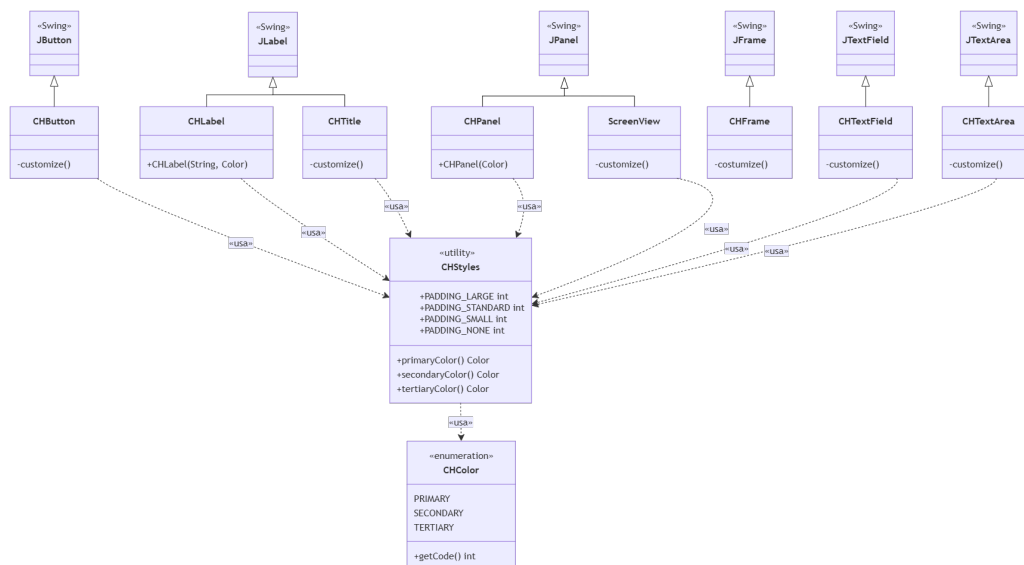


Figura 2.1: Figura 2.7

2.2.5 Caricamento lazy e cache delle immagini delle carte (ImageResolver)

Problema

Caricare un'immagine da risorsa (classpath) e scalarla alla dimensione standard ($CARD_WIDTH \times CARD_HEIGHT$) è un'operazione costosa (I/O + elaborazione grafica); la stessa immagine può essere richiesta molte volte: ogni volta che una carta compare in una view, e ogni volta che una view viene ridisegnata dopo un *update*(*) del controller (es. mano/playfield che si aggiornano ad ogni mossa). Perciò va evitato di ripetere il lavoro di caricamento/scaling per lo stesso file.

Soluzione

SimpleCardImageResolver incapsula da solo il caricamento + scaling, mentre **CachingCardImageResolver** aggiunge la cache (**ConcurrentHashMap**<**String**, **ImageIcon**>, popolata via *computeIfAbsent()*) sopra una qualunque implementazione dell'interfaccia **CardImageResolver** ricevuta nel costruttore. La prima richiesta per una data carta/dorso viene davvero calcolata dal resolver decorato; le richieste successive per la stessa chiave restituiscono l'istanza già cachata, senza chiamare **CardImageResolver**.

ImageResolver assembla staticamente la catena (**CachingCardImageResolver** che avvolge **SimpleCardImageResolver**) una sola volta e la espone con un'API statica comoda per le view, che non devono gestire/passare in giro

un'istanza.

Pattern

Proxy: La struttura è quella canonica del pattern in cui **CardImageResolver** è il Subject: l'interfaccia comune a RealSubject e Proxy; **SimpleCardImageResolver** è il RealSubject: esegue il lavoro reale; e **CachingCardImageResolver** è il Proxy che in questo caso ha una doppia natura:

- Caching: perché evita di ripetere un'operazione costosa (I/O più elaborazione grafica);
- Virtuale: perché rimanda la creazione reale dell'immagine al primo utilizzo effettivo, senza che il RealSubject venga mai interrogato più di una volta per chiave.

La classe **ImageResolver** non fa parte della struttura del pattern in senso stretto: è una facciata statica che assembla una sola volta la catena **CachingCardImageResolver(SimpleCardImageResolver)** e la espone alle view (**PlayfieldPanelImpl**, **PlayerPanelImpl**) tramite i due soli metodi statici *resolve()* e *resolveBack()*, cosicché nessuna view debba costruire né conoscere la catena Proxy→RealSubject sottostante.

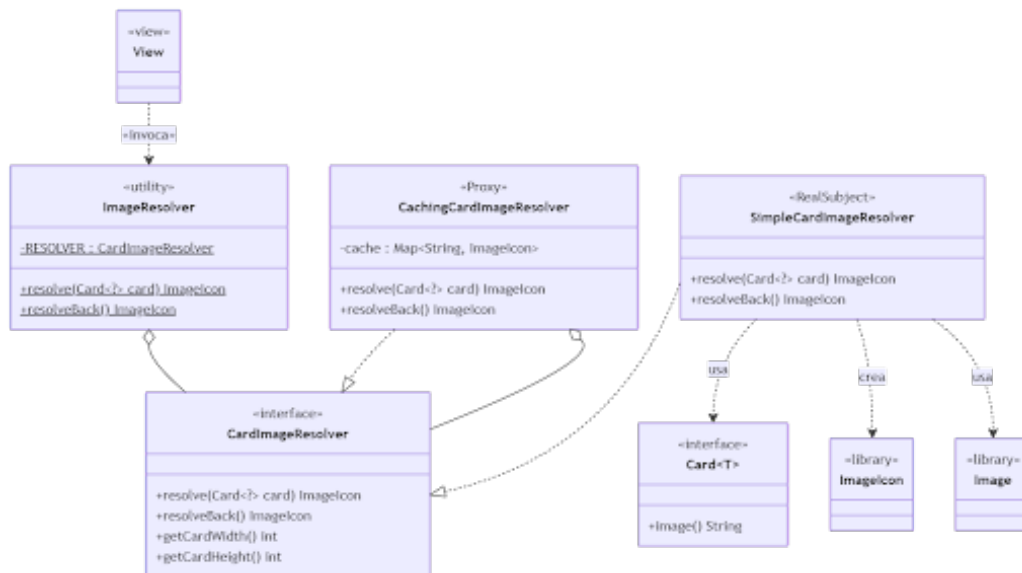
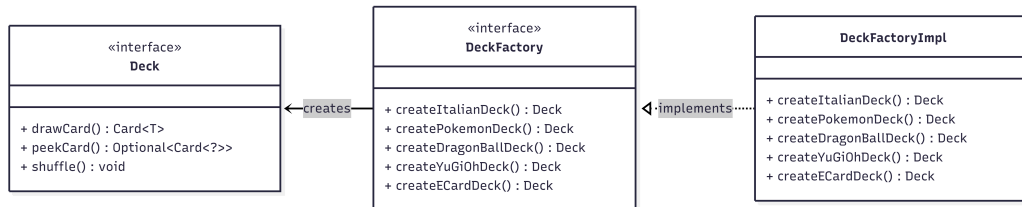


Figura 2.2: Figura 2.8

2.2.6 Creazione e caricamento dei mazzi (Portolani)



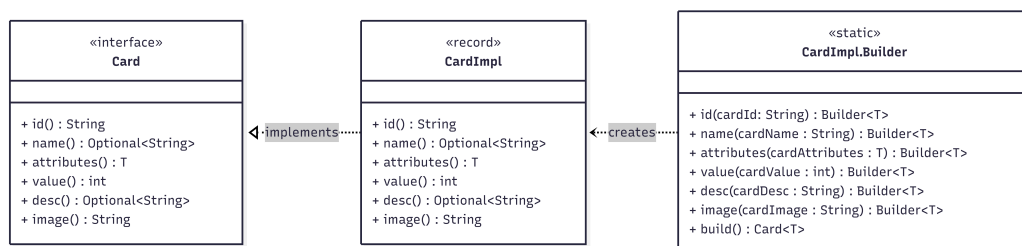
Problema

L'applicazione deve supportare diverse tipologie di mazzi di carte, caratterizzate da differenti contenuti. In particolare, alcuni vengono generati programmaticamente, mentre altri vengono caricati da risorse esterne. Senza un'opportuna astrazione, i componenti che richiedono un mazzo dovrebbero conoscere i dettagli relativi alla sua costruzione. Ciò aumenterebbe l'accoppiamento tra i componenti dell'applicazione.

Soluzione

Per separare la creazione dei mazzi dal loro utilizzo, la soluzione adotta il *Factory pattern* realizzato tramite l'interfaccia **DeckFactory**, da cui dipendono i client, i quali possono richiedere direttamente il prodotto desiderato, senza conoscere il processo necessario per costruirlo grazie a **DeckFactoryImpl**, a cui è affidata la costruzione concreta. In questo modo, *DeckFactoryImpl* incapsula tali dettagli mantenendoli separati dai componenti che utilizzano un mazzo e pertanto favorendo il riuso e futura estensione. Inoltre, il prodotto restituito dalle operazioni della factory è rappresentato dall'astrazione **Deck**. In questo modo i client dipendono dall'interfaccia del prodotto anziché dalla sua implementazione concreta. L'estensione del sistema con ulteriori tipologie di mazzo può inoltre essere affrontata concentrando le modifiche nella componente responsabile della loro creazione, evitando che i client debbano conoscere direttamente le nuove modalità di caricamento o costruzione.

2.2.7 Costruzione flessibile delle carte



Problema

CardImpl rappresenta l'implementazione dell'interfaccia *Card*, a cui è affidata la costruzione delle carte. Siccome *CardImpl* è sostanzialmente una Data Class, essa può essere definita come record. I record in Java forniscono un costruttore canonico predefinito, ma resta comunque il problema del fatto che le carte presentano più proprietà, alcune opzionali. Ciò può comportare la mancanza di leggibilità nel caso in cui i client devono costruire una carta, perché devono sapere a priori quali sono i parametri e in quale sequenza devono essere passati al costruttore.

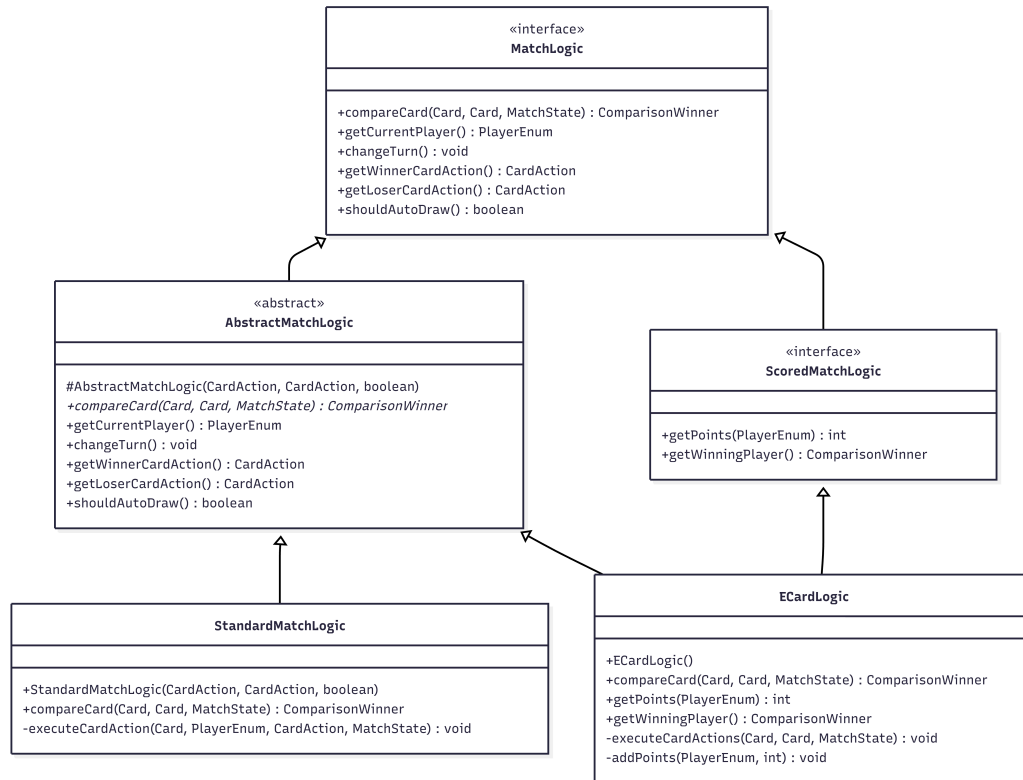
Soluzione

Per una costruzione più flessibile e leggibile è stato utilizzato il *Builder pattern* realizzato tramite la classe statica e innestata **CardImpl.Builder**. I vantaggi di questa soluzione sono dunque:

- **Chiarezza:** il Builder offre un modo più leggibile per costruire una carta.
- **Flessibilità:** il Builder consente di impostare i campi in modo incrementale, il che è molto utile, data la presenza di campi opzionali, e permette di costruire una carta in modo non lineare.
- **Immutabilità:** anche se si utilizza un Builder per impostare i campi, l'oggetto *CardImpl* risultante rimane immutabile, in linea con i principi dei record.

L'aggiunta del Builder, pertanto, rende la costruzione di *CardImpl* più versatile e il codice più pulito e più facile da gestire, senza sacrificare l'immutabilità, semplicità ed estensibilità che i record offrono.

2.2.8 Gestione della logica di gioco (Ciliberti)



Problema

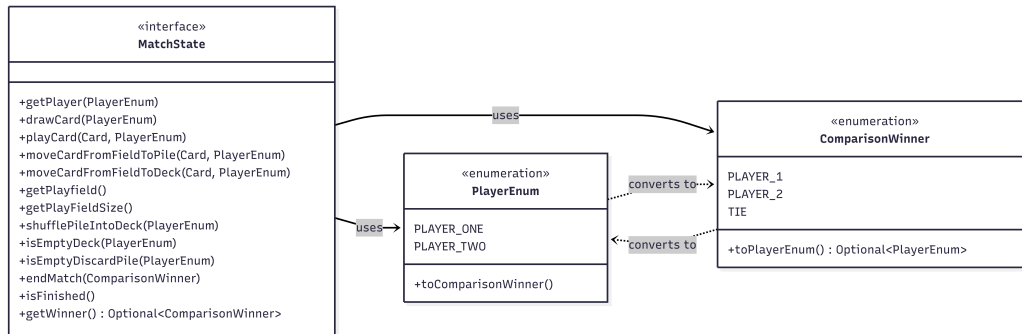
La logica di E-Card non dovrebbe essere creata da zero, ma dovrebbe basarsi sugli stessi strumenti a cui i giocatori hanno accesso nella modalità Custom Play.

Soluzione

Si è deciso di applicare lo *Strategy Pattern*: L'interfaccia **MatchLogic** definisce il contratto della logica. La classe astratta **AbstractMatchLogic** fa da strategy, implementando tutti i metodi di *MatchLogic* tranne *CompareCard*, che viene definito come astratto. Le due estensioni **StandardMatchLogic** e **ECardLogic** implementano questo metodo in modo diverso, poichè E-Card si basa su delle regole stile carta-sasso-forbici.

Poichè E-Card si basa su un sistema a punti per determinare il vincitore, è stata creata un'interfaccia **ScoredMatchLogic** che estende *MatchLogic*, dedicata alle modalità di gioco che implementano la gestione dei punti. *ECardLogic* implementa quindi *ScoredMatchLogic*, oltre ad estendere *AbstractMatchLogic*.

2.2.9 Incapsulamento del Player



Problema

Molte delle classi nel progetto hanno bisogno di sapere a quale dei due player corrisponde un'azione che deve essere effettuata, senza il bisogno di possedere i dati relativi al suddetto player.

Ai fini del corretto incapsulamento si vuole quindi evitare di passare in giro tutto il player quando non necessario

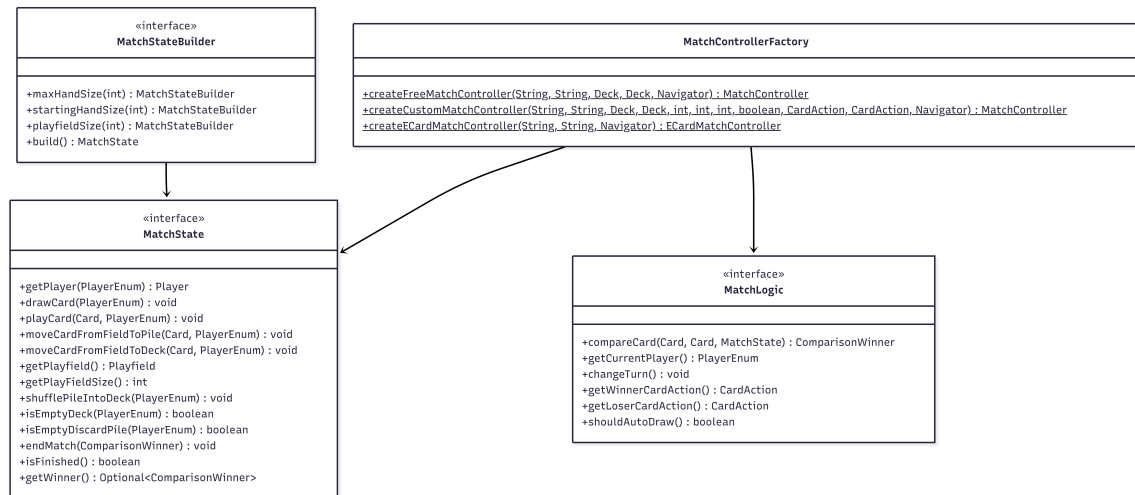
Soluzione

Si è deciso di creare una *enum* chiamata **PlayerEnum**, che viene passata come parametro ai metodi che non hanno bisogno di conoscere informazioni sul player, ma vogliono solo sapere su chi dei due deve essere effettuata l'azione (ad esempio, per dire alla GUI di aggiornare la parte della view relativa al giocatore 1 o al giocatore 2).

In *MatchState* è presente il metodo *getPlayer(PlayerEnum)* che permette di ricevere l'effettivo player corrispondente alla enum, e può essere usato dalle classi che hanno accesso a *MatchState* (*MatchController*, e *MatchLogic* che lo chiede come parametro in *compareCard*) quando hanno bisogno di accedere effettivamente ai dati del giocatore.

La enum **ComparisonWinner** è un'altra enum con uno scopo simile, ma che viene usata specificatamente per la gestione del vincitore di una comparison, e in quanto tale ha anche un valore *TIE*. Le due enum hanno comunque dei metodi per la conversione da una all'altra.

2.2.10 Inizializzazione della partita



Problema

Per iniziare la partita, è necessario creare lo stato e la logica di gioco. La creazione dello stato e il tipo di logica che viene utilizzato dipendono da un quantitativo molto grande di parametri.

Soluzione

Si è deciso di creare un *builder* per *MatchState* sotto forma di **MatchStateBuilderImpl**. Questo builder prende nel costruttore dei parametri considerati obbligatori (i nomi e i mazzi dei giocatori), e ha dei metodi per settare dei parametri opzionali (dimensione iniziale e massima della mano, dimensione massima del playfield) che eventualmente vengono settati a dei valori di default. Il metodo *build()* usa quindi questi parametri per creare i giocatori e lo stato di gioco, restituendo quindi un *MatchState*.

Lo stato e la logica vengono poi usati per creare il *MatchController*, lavoro di cui si occupa la *factory* **MatchControllerFactory**.

MatchControllerFactory ha dei metodi che prendono in input i parametri necessari, creano un *MatchState* adatto usando appunto *MatchStateBuilder*, creano una logica adatta al tipo di partita che si sta creando, e restituiscono un *MatchController* creato usando il *Navigator* fornito, e i suddetti stato e logica appena creati. In questo modo si evita che sia il Controller e prendere attivamente parte alla creazione di parti del model.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Per verificare la correttezza delle funzionalità principali del modello è stato realizzato un insieme di test automatizzati utilizzando JUnit. I test riguardano principalmente le componenti responsabili della gestione delle carte e dello stato della partita, in particolare `DeckImpl`, `HandImpl`, `DiscardPileImpl`, `PlayerImpl`, `PlayfieldImpl`, `MatchStateImpl` e `DeckFactoryImpl`.

La suite di test verifica sia i normali comportamenti delle componenti, come l'aggiunta, la rimozione e il trasferimento delle carte, sia la gestione di condizioni limite, come collezioni vuote o piene. Sono inoltre verificate le eccezioni previste dal modello e la corretta creazione dei diversi tipi di mazzo.

Particolare attenzione è stata dedicata ad alcuni casi di regressione individuati durante lo sviluppo, tra cui la gestione del tentativo di pescare una carta con la mano piena e la rimozione di carte uguali appartenenti a giocatori differenti. In quest'ultimo caso i test verificano che l'operazione venga effettuata esclusivamente sul giocatore specificato.

MatchStateBuilder è stato testato in maniera abbastanza veloce, verificando che lo stato di gioco venisse creato correttamente, sia fornendo i parametri opzionali, sia lasciandoli allo stato di default.

Per quanto riguarda la logica si è verificato che le card actions e l'autodraw venissero effettuate correttamente, oltre ad aver testato che il metodo *compareCard()* funzionasse correttamente (specialmente nel caso di *ECardLogic*).

MatchControllerFactory non è stata testata poichè i suoi metodi si limitano ad istanziare oggetti, ed eventuali problemi sono quindi stati testati nei test dei singoli oggetti.

La navigazione tra le schermate è stata testata verificando che il navigatore

mostrasse la schermata fornita, salvandola nella sua cache temporanea e rimuovendola durante la navigazione successiva. Inoltre, è stato testato il fatto che una volta mostrata la schermata Home essa resti sempre a disposizione. Per quanto riguarda il controller della partita, sono stati sottomessi al testing gli aggiornamenti del model e view ad ogni azione prevista in una partita, verificando la corretta notifica verso la view davanti ad un tentativo di esecuzione di un'azione invalida. Per riuscire a realizzare il testing automatizzato di questa sezione è stato necessario creare delle classi ad hoc che non eseguissero delle azioni bloccanti (pop-ups di JSwing). Altrimenti non sarebbe stato possibile eseguire nessun testing automatizzato.

Per il controller della creazione della partita, invece, si è deciso di limitare il testing automatico alla sua navigazione e alla verifica dei dati recuperati dal model che vengono inoltrati alla view. La motivazione deriva del fatto che ogni richiesta di creazione di partita conduce ad un'azione bloccante: pop-up di modulo invalido per richiesta invalida; pop-up di inizio di turno per richiesta valida.

Tutti i test sono completamente automatizzati e vengono eseguiti tramite Gradle, ad eccezione di quelli dell'interfaccia visiva che sono stati effettuati in maniera manuale, testando che la gui non avesse particolari problemi su schermi di risoluzione diversa, oppure ridimensionando la finestra. Al termine dello sviluppo, l'intera suite di test risulta superata, garantendo la verifica automatica dei comportamenti considerati rilevanti per la parte di modello sviluppata.

3.2 Note di sviluppo

3.2.1 Basilisco

Utilizzo di generici:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/main/java/it/unibo/cardhub/controller/impl/AbstractMatchController.java#L28>

Utilizzo di Lambda Expressions:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/test/java/it/unibo/cardhub/controller/MatchControllerTest.java#L185>

Utilizzo di reflection:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/test/java/it/unibo/cardhub>

[b/controller/MatchControllerTest.java#L184](#)

Utilizzo di Optional e dei suoi metodi:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/main/java/it/unibo/cardhub/view/impl/PlayfieldPanelImpl.java#L172>

Utilizzo di BiConsumer<T, U>:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/main/java/it/unibo/cardhub/view/impl/PlayfieldPanelImpl.java#L204>

Utilizzo di Function<T, R>:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/main/java/it/unibo/cardhub/view/impl/PlayfieldPanelImpl.java#L312>

Utilizzo di Runnable:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/main/java/it/unibo/cardhub/view/impl/PlayfieldPanelImpl.java#L313>

3.2.2 Portolani

Dichiarazione di un'interfaccia con un tipo generico:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/660ae1e253781dd8b9f70280a4a1648c6a659fba/src/main/java/it/unibo/cardhub/model/domain/api/Card.java#L10>

Utilizzo di lambda expressions:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/a52b9a11b657df8a2e7c036547f368948c9b5f81/src/main/java/it/unibo/cardhub/model/domain/impl/DiscardPileImpl.java#L54>

Utilizzo di Stream:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/a52b9a11b657df8a2e7c036547f368948c9b5f81/src/main/java/it/unibo/cardhub/model/domain/impl/PlayfieldImpl.java#L103>

Utilizzo di Optional:

Optional, Permalink: <https://github.com/FlavioCiliberti/00P25-CardHub/blob/2cf40de6401856eb4f0b83200ec788eda69a0c8/src/main/java/it/unibo/cardhub/view/impl/PlayfieldPanelImpl.java#L172>

Utilizzo di Builder dalla libreria Lombok:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/a52b9a11b657df8a2e7c036547f368948c9b5f81/src/main/java/it/unibo/cardhub/model/domain/impl/CardImpl.java#L20>

Definizione di un metodo generico con l'utilizzo di Stream e Yaml dalla libreria SnakeYAML:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/a52b9a11b657df8a2e7c036547f368948c9b5f81/src/main/java/it/unibo/cardhub/io/impl/DeckFactoryImpl.java#L121>

3.2.3 Ciliberti

Utilizzo di Optional:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/bcad25efcf6c475f32f678b33487c898b6cc2a49/src/main/java/it/unibo/cardhub/view/impl/PlayerPanelImpl.java#L194>

Utilizzo di lambda expressions:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/c357430f31567819831f9152ea41d7bd5de9aabd/src/main/java/it/unibo/cardhub/view/impl/CreateMatchViewImpl.java#L394>

Utilizzo di campi e metodi all'interno di una enum:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/bcad25efcf6c475f32f678b33487c898b6cc2a49/src/main/java/it/unibo/cardhub/model/domain/attributes/ECardEnum.java#L6>

Utilizzo di *instanceof*:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/bcad25efcf6c475f32f678b33487c898b6cc2a49/src/main/java/it/unibo/cardhub/view/impl/PlayerPanelImpl.java#L236>

Utilizzo di variable-length arguments:

<https://github.com/FlavioCiliberti/00P25-CardHub/blob/bcad25efcf6c475f32f678b33487c898b6cc2a49/src/main/java/it/unibo/cardhub/view/impl/CreateMatchViewImpl.java#L440>

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Basilisco

Il mio ruolo è stato principalmente definito dal lavoro sull'integrazione tra GUI e logica, garantendo l'indipendenza fra esse.

Nello specifico le mie responsabilità sono state:

- Implementazione della architettura MVC.
- Progettazione e sviluppo dei controller.
- Progettazione delle api delle view.
- Progettazione dello scheletro dell'app (schermate e navigazione fra esse).
- Creazione dei componenti fondamentali per la creazione delle GUI.
- Sviluppo della GUI della schermata Home.

Come punti di forza menzionerei il corretto approccio progettuale a step (dallo macro allo micro), la scrittura di codice chiaro e sintetico seguendo il principio di singola responsabilità (ottenuto dopo progressivi refactoring), l'uso delle buone pratiche di programmazione orientata agli oggetti e l'uso consapevole delle feature avanzate di Java.

Tra gli aspetti che, a mio parere, possono essere oggetto di autocritica, c'è quella di aver concepito un progetto forse troppo ambizioso. Se avessimo deciso di sviluppare meno funzionalità, saremmo riusciti a dedicare più tempo e attenzione alla qualità di ogni classe, splittando le responsabilità delle classi

più lunghe e facendo altri refactoring mirati.

Un'altro aspetto migliorabile da menzionare è che, sebbene abbiamo avuto un approccio al lavoro di scrittura del codice molto dinamico, apportando piccoli cambiamenti anche su sezioni di altri membri (previo consenso del responsabile), ciò potrebbe aver avuto un impatto negativo sull'omogeneità di alcune sezioni. Ognuno, infatti, ha il suo stile e approccio diverso alla scrittura del codice.

In conclusione, ritengo che, nonostante il ritiro di un membro del gruppo e la scelta di un progetto di complessità non trascurabile, siamo riusciti a sviluppare un'applicazione funzionante e di buona qualità dal punto di vista della programmazione, seppur ancora migliorabile attraverso alcuni interventi di refactoring. Il progetto ci ha inoltre permesso di acquisire una maggiore padronanza di Java e, più in generale, dei principi della programmazione ad oggetti.

Per questo motivo, mi piacerebbe continuare lo sviluppo dell'applicazione, iniziando dal completamento delle funzionalità rimaste incomplete e dall'integrazione di un database.

4.1.2 Portolani

Il mio ruolo era principalmente lo sviluppo del modello di dominio, in particolare gli elementi statici che compongono una partita di un gioco di carte, e caricamento delle carte. Mi sono inoltre occupato della GUI di alcune schermate dell'applicazione, tuttavia ho dovuto scartare questa parte del mio lavoro per tempistica limitata e ristrutturazione del progetto, dovuta all'allontanamento di un membro del gruppo. Infine ho scritto e mantenuto test JUnit relativi alle componenti del modello e creazione delle carte, nonché lavorato sulla qualità del codice e sul refactoring.

A mio parere, tra i punti di forza del mio contributo al progetto sono stati: l'attenzione alla correttezza dello stato del modello, l'utilizzo dei test per individuare regressioni, refactoring in linea con i principi delle buone pratiche di programmazione orientata agli oggetti e infine l'uso consapevole delle feature avanzate di Java. Tra i punti di debolezza invece, menzionerei il fatto che il nostro progetto non è riuscito come è stato pianificato dall'inizio per le problematiche riscontrate e perciò abbiamo dovuto prendere varie deviazioni durante lo sviluppo per arrivare a un buon compromesso tra qualità e tempo limitato. Alcune parti del codice, ad esempio, avrebbero potuto essere progettate in modo più generale fin dall'inizio.

In conclusione, penso che abbiamo realizzato un progetto complessivamente

di qualità e funzionante: pertanto io avrei piacere di riutilizzarlo o riadattarlo per lavori futuri, ad esempio nei progetti di Basi di dati, Tecnologie web e altri futuri progetti della mia carriera accademica. Il fatto che l'estensibilità è stato uno dei requisiti del progetto fin dall'inizio rafforza questa possibilità: ad esempio, si potrebbe aggiungere nuovi giochi oltre a quelli già implementati, migliorare la gestione delle immagini delle carte ed eventualmente rendere il sistema di caricamento dei mazzi ancora più estendibile, così da poter aggiungere un nuovo mazzo senza modificare il controller.

4.1.3 Ciliberti

Il mio ruolo principale è stato lo sviluppo della logica e degli elementi necessari al collegamento del Controller con il Model (MatchControllerFactory e MatchStateBuilder principalmente), e altri contributi minori sempre relativi al model. Mi sono anche occupato di una porzione della GUI, nello specifico la schermata di creazione della partita, lo scheletro esterno della schermata della partita e i pannelli relativi ai player (quelli che mostrano il deck e la mano).

Il punto di forza principale a mio parere è la comprensibilità e correttezza del codice da me scritto nel model, a mio parere abbastanza intuitivo da leggere. Come principale punto di debolezza metterei invece la mia poca dimistichezza con la creazione delle GUI in Swing, che ha portato a un codice poco leggibile nelle parti della GUI.

Sono dispiaciuto del fatto che la perdita di un membro abbia impedito di raggiungere tutti gli obiettivi che ci eravamo prefissati, oltre ad aver causato un leggero ritardo dovuto alla spartizione del lavoro extra del quarto membro. Nonostante ciò ritengo comunque che il progetto abbia raggiunto uno stato di cui possiamo ritenerci soddisfatti, poichè mantiene comunque il suo scopo principale di simulatore di carte, oltre ad essere scritto in modo da poter essere eventualmente espanso in futuro.

In conclusione, mi piacerebbe molto, in un eventuale futuro, poter concludere questo progetto aggiungendo tutte le feature che non siamo riusciti ad introdurre.

Appendice A

Guida utente

All'apertura del gioco si preme il tasto **New Match** per creare una partita, si inseriscano i nomi e i deck dei giocatori e si scelga la modalità di gioco (*Free Play* e *Custom Rules* sono identici con la differenza che la seconda permette di personalizzare alcuni dei parametri di gioco).

Durante la partita, oltre ai pulsanti presenti a schermo, autoesplicativi, è possibile effettuare alcune azioni cliccando le carte o delle zone di gioco. Queste azioni sono elencate nel *README* presente nella repository.

Nel caso si decida di giocare ad **E-Card**, le regole di gioco sono le seguenti:

1. Il giocatore 1 sceglie una carta, che verrà giocata a faccia in giù.
2. Il giocatore 2 sceglie una carta, che verrà giocata a faccia in giù.
3. Le carte vengono scoperte, e i giocatori guadagnano punti secondo le seguenti regole:
 - L'imperatore batte il cittadino per un punto.
 - Il cittadino batte lo schiavo per un punto.
 - Lo schiavo batte l'imperatore per 3 punti.
4. Si continua il gioco finchè i giocatori non esauriscono le carte, viene poi dichiarato vincitore il giocatore con più punti.

Appendice B

Esercitazioni di laboratorio

B.1 tobias.basilisco@studio.unibo.it

- Laboratorio 06: <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284056>
- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p285023>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286136>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287094>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288483>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289670>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290690>